

SIMATS ENGINEERING

CO3 - ASSESSMENT TOOL 1

Case Study

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA0408

COURSE OUTCOME COVERED

CO3: Analyze and apply convolutional neural network architectures, including convolutional layers, filters, parameter sharing, regularization, AlexNet and ResNet, to develop solutions for image classification and real-world computer vision applications. (L4)

CO Weightage: 25%

Assessment Tool Weightage: 25%

QUESTIONS:

Total Marks: 25

Case Study 1: CNN for Automated Medical Image Classification

A healthcare organization wants to develop a deep learning system to automatically classify chest X-ray images into **normal** and **disease-affected** categories. The dataset contains thousands of X-ray images with variations in size, brightness and image quality. A CNN model is proposed because it can automatically learn important visual features such as edges, textures, and abnormal regions without manually designing features.

Question:

1. Design and explain a suitable **CNN architecture** for this medical image classification system.
2. Describe the **motivation for using CNNs** and explain the role of **convolutional layers, filters, pooling layers, and fully connected layers** in the proposed architecture.
3. Explain how **parameter sharing** reduces the number of trainable parameters compared with a fully connected network.

4. Discuss suitable **regularization techniques such as dropout, data augmentation and batch normalization** to reduce overfitting.
5. Finally, explain whether **AlexNet or ResNet** would be more appropriate for this application and justify your choice based on feature learning, network depth, computational requirements and classification performance.

Case Study 2: CNN-Based Autonomous Vehicle Object Recognition

An autonomous vehicle company is developing a vision system that must identify **cars, pedestrians, traffic signs, bicycles, and road obstacles** from images captured by cameras mounted on the vehicle. The system must recognize objects under different lighting conditions, viewing angles, distances and road environments. The development team is considering **AlexNet and ResNet** architectures for building the CNN-based recognition system.

Question:

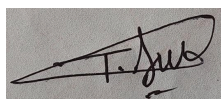
1. Analyze how a **CNN architecture** can be designed for this autonomous vehicle application.
2. Explain the purpose of the **input, convolution, activation, pooling, and fully connected/output layers** and describe how convolutional **filters progressively learn low-level and high-level features**.
3. Explain the importance of **parameter sharing** in reducing computational complexity and improving translation-related feature detection.
4. Discuss the possible problem of **overfitting** and recommend suitable regularization methods.
5. Compare **AlexNet and ResNet** in terms of architecture, depth, parameter efficiency, training difficulty, vanishing-gradient problems and feature-learning capability.
6. Finally, recommend the most suitable architecture for the autonomous vehicle system and justify your decision based on **accuracy, computational cost and real-time application requirements**.

Code of Conduct:

I T. Narasimha (192425233) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Case Study 1

1. Design and explain a suitable CNN architecture for the medical image classification system.

A suitable CNN architecture for classifying chest X-ray images into normal and disease-affected categories can be designed as follows:

Input Image → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Convolution → ReLU → Pooling → Fully Connected Layer → Dropout → Output Layer

Layer	Function	Layer
Input Layer	Accepts resized chest X-ray images, e.g., $224 \times 224 \times 1$	Input Layer
Conv2D	Extracts edges, textures and other visual patterns	Conv2D
ReLU	Introduces non-linearity	ReLU
Max Pooling	Reduces spatial dimensions	Max Pooling
Conv2D	Learns more complex features	Conv2D
ReLU	Activates important features	ReLU
Max Pooling	Reduces computational requirements	Max Pooling
Conv2D	Detects high-level abnormal patterns	Conv2D
ReLU	Non-linear feature transformation	ReLU
Max Pooling	Further reduces feature-map size	Max Pooling
Fully Connected Layer	Combines extracted features	Fully Connected Layer
Dropout	Reduces overfitting	Dropout
Output Layer	Produces probability for normal/disease classes	Output Layer

The X-ray images should first undergo preprocessing, including resizing, normalization and possibly contrast enhancement. The convolutional layers progressively learn meaningful patterns such as edges, lung textures and abnormal regions. The final classification layer can use sigmoid activation for binary classification.

2. Explain the motivation for using CNNs and the role of convolutional layers, filters, pooling layers and fully connected layers.

CNNs are suitable for medical image classification because they can automatically learn visual features directly from images without requiring manually designed features. This is particularly useful for X-rays because abnormalities may appear as differences in texture, shape, intensity or structure. The case study specifically identifies edges, textures and abnormal regions as important features learned by CNNs.

Convolutional Layers

Convolutional layers apply learnable filters to the input image. They detect important local patterns and generate feature maps.

For example:

- Early layers → edges and contours
- Middle layers → textures and shapes
- Deeper layers → complex anatomical or abnormal patterns

Filters

A filter, or kernel, is a small matrix of learnable weights that slides across the image.

For example, a filter may learn to detect:

- Horizontal/vertical edges
- Curves
- Textures
- Region boundaries
- Disease-related patterns

During training, the CNN automatically learns the most useful filter values.

Pooling Layers

Pooling reduces the spatial dimensions of feature maps while retaining important information.

Max pooling is commonly used because it selects the maximum value from a local region.

Benefits:

- Reduces computation

- Reduces memory requirements
- Provides some translation tolerance
- Helps control overfitting
- Fully Connected Layers

The final feature maps are converted into a vector and passed to fully connected layers. These layers combine the learned features and make the final classification decision.

For binary classification, the final layer can produce:

Normal $\rightarrow 0$

Disease affected $\rightarrow 1$

3. Parameter Sharing:

Parameter sharing means that the same convolutional filter weights are reused across different spatial locations of an image.

Consider a 224×224 grayscale image.

A fully connected neuron connected to every pixel would require:

$$224 \times 224 = 50,176$$

weights for just one neuron, excluding the bias.

In a CNN, suppose we use a 3×3 filter. The filter requires only:

$$3 \times 3 = 9$$

weights, plus one bias.

The same 9 weights are then applied across the entire image.

Therefore, instead of learning separate weights for every image location, the CNN learns a small set of filters and shares those parameters across the image.

This provides two major advantages:

1. **Fewer trainable parameters** – reduces memory and computational requirements.

- 2. Translation-related feature detection** – the same feature can be detected regardless of where it occurs in the image.

Thus, parameter sharing makes CNNs much more efficient than fully connected networks for image processing.

4. Regularization Techniques:

The medical dataset contains variations in image size, brightness and image quality, which can make model training challenging. Regularization is therefore important to prevent overfitting.

a) Dropout

Dropout randomly deactivates a proportion of neurons during training.

For example, with a dropout rate of 0.5, approximately 50% of selected neurons are temporarily ignored during each training iteration.

Advantages:

- Prevents neurons from becoming overly dependent on each other
- Improves generalization
- Reduces overfitting

b) Data Augmentation

Data augmentation artificially creates variations of existing training images.

For chest X-rays, suitable transformations can include:

- Small rotations
- Translation
- Scaling
- Brightness variation
- Contrast variation
- Horizontal flipping where medically appropriate

This helps the CNN become more robust to differences in image acquisition and quality.

c) Batch Normalization

Batch normalization normalizes activations within mini-batches during training.

Benefits include:

- More stable training
- Faster convergence
- Improved gradient flow
- Some regularization effect

Therefore, combining data augmentation + batch normalization + dropout provides a strong strategy for reducing overfitting.

5. AlexNet vs ResNet Recommendation:

Factor	AlexNet	ResNet
Architecture	Relatively conventional CNN	Uses residual/skip connections
Depth	Shallower	Can be significantly deeper
Feature learning	Good	More powerful hierarchical feature learning
Vanishing gradients	More susceptible	Skip connections help reduce the problem
Training deep networks	More difficult	Easier
Computational requirement	Generally lower	Generally higher depending on variant
Classification capability	Good baseline	Usually stronger for complex visual tasks

Case Study 2

1. CNN Architecture Design:

A CNN-based autonomous vehicle vision system can be designed using the following pipeline:

Camera Image → Preprocessing → Convolution → ReLU → Pooling → Deeper Convolution Layers → Feature Extraction → Fully Connected/Detection Head → Object Classification

The system receives images from vehicle-mounted cameras.

Proposed architecture

1. Input Layer

- Receives RGB camera images.
- Images can be resized to a fixed resolution.

2. Convolutional Layers

- Extract visual features.
- Early layers identify edges and basic shapes.
- Deeper layers identify objects and complex patterns.

3. Activation Functions

- ReLU introduces non-linearity.
- Allows the network to learn complex relationships.

4. Pooling Layers

- Reduce spatial dimensions.
- Reduce computational requirements.

5. Deep Feature Extraction

- Learns high-level representations of cars, pedestrians, signs and obstacles.

6. Classification/Detection Layer

- Determines the object category.
- For a practical autonomous vehicle system, an object-detection head would additionally predict object locations using bounding boxes.

The architecture should also be robust to changes in lighting, distance, viewing angle and road conditions.

2. Layer Purposes and Feature Hierarchy:

Input Layer

The input layer receives images captured by vehicle-mounted cameras.

For example:

Input=224×224×3

Where:

- 224 = image height
- 224 = image width
- 3 = RGB channels

Convolution Layer

Convolutional layers apply filters to the image and extract local features.

The first layers may learn:

Edges → Corners → Textures → Shapes

Deeper layers can learn:

Wheels → Vehicle shapes → Human silhouettes → Traffic signs → Complex objects

Activation Layer

ReLU is commonly used:

$\text{ReLU}(x) = \max(0, x)$

It introduces non-linearity, allowing the CNN to learn complex patterns.

Pooling Layer

Pooling reduces the spatial dimensions of feature maps.

For example, 2×2 max pooling can reduce the spatial dimensions approximately by half.

This:

- Reduces computation
- Reduces memory usage
- Retains important features
- Provides some robustness to small translations

Fully Connected/Output Layer

The final layers combine extracted features and generate predictions.

For object classification, possible classes include:

- Car
- Pedestrian
- Bicycle
- Traffic sign
- Road obstacle.

3. Importance of Parameter Sharing:

Parameter sharing is extremely important in autonomous vehicle vision because the same object can occur at different locations in an image.

For example, a car may appear:

- On the left side
- In the center
- On the right side

A CNN filter can detect the same car-related feature regardless of its position.

Instead of learning separate weights for every pixel location, a CNN reuses the same filter.

Benefits

1. Reduced parameters

A small convolutional kernel requires far fewer parameters than a fully connected layer connected to every pixel.

2. Lower computational complexity

Fewer parameters reduce memory requirements and computational operations.

3. Translation-related feature detection

The same filter can identify an edge, wheel or sign regardless of its position.

4. Better generalization

The model learns reusable visual patterns instead of location-specific patterns.

Therefore, parameter sharing is essential for efficient real-time image processing in autonomous vehicles.

4. Overfitting and Regularization:

Overfitting occurs when a CNN learns the training images too closely but performs poorly on unseen road scenes.

This can happen because:

- The network contains many parameters.
- Training data may not cover every possible road condition.
- Certain environments may be overrepresented.
- The model may memorize rather than generalize.

Recommended techniques

1. Data Augmentation

Generate different training examples using:

- Brightness changes
- Contrast changes
- Rotation
- Scaling
- Cropping
- Translation
- Weather-related transformations where appropriate

This makes the model robust to different environmental conditions.

2. Dropout

Dropout randomly disables neurons during training and reduces dependence on specific neurons.

3. Batch Normalization

Batch normalization stabilizes activations and can improve training and generalization.

4. Early Stopping

Training can be stopped when validation performance stops improving.

5. Weight Regularization

L2 regularization can penalize excessively large weights and help prevent overfitting.

5. AlexNet vs ResNet Comparison:

Feature	AlexNet	ResNet
Architecture	Traditional deep CNN	CNN with residual/skip connections
Depth	Relatively shallow	Can be very deep
Main innovation	Demonstrated strong deep CNN performance for image recognition	Residual learning
Parameter efficiency	Less efficient for modern deep feature learning	Better capability relative to depth
Training difficulty	Easier than very deep networks but limited in depth	Skip connections make deep training easier
Vanishing gradients	More susceptible in deeper configurations	Residual connections help address gradient degradation
Feature learning	Learns useful visual features	Learns richer hierarchical features
Computational cost	Generally lower	Usually higher for larger variants
Accuracy potential	Good	Generally higher
Modern computer vision suitability	Mainly a baseline/historical architecture	More suitable for complex vision applications

6. Recommended Architecture:

ResNet would be the better choice between AlexNet and ResNet for an autonomous vehicle vision system.

Justification

1. Higher feature-learning capability

Autonomous vehicles must recognize many different objects under challenging conditions. ResNet's deeper architecture can learn more complex hierarchical features.

2. Better handling of deep networks

Residual connections improve gradient flow and make deep network training more practical.

3. Better classification performance

ResNet generally provides stronger image recognition performance than older architectures such as AlexNet, particularly for complex visual datasets.

4. Robustness

The deeper feature representations can help distinguish visually similar objects and handle variations in viewpoint, scale and appearance.

5. Computational consideration

A very deep ResNet can be computationally expensive. Therefore, the system should select an appropriate ResNet variant and optimize it for deployment, potentially using techniques such as model compression or hardware acceleration.

Final Decision

For an autonomous vehicle system where accuracy and robust feature extraction are critical, ResNet is preferred over AlexNet. However, because autonomous vehicles require real-time processing, the chosen ResNet variant must balance accuracy, latency, memory consumption and computational cost.